Personal Note of Fourier Neural Operator for Parametric Partial Differential Equations

Authors：Zongyi Li, Nikola Borislavov Kovachki , Kamyar Azizzadenesheli, Burigede liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar

Affiliations：Caltech, Purdue

Published：ICLR 2021

Background

- Traditional methods, whether finite element or finite difference, involve solving equations in a discrete space. This inevitably leads to a resolution tradeoff: a coarser resolution grid is faster but less accurate, while a finer resolution grid is slower but more precise. **For traditional solvers, higher resolution means slower computation.** This tradeoff signifies that traditional methods require more computational cost to achieve higher accuracy, and this cost includes both time and money. Designing an algorithm through modifying traditional methods that is both highly accurate and low in computational cost is challenging. In my opinion, the AI for PDE approach has the potential to bridge this gap, achieving faster and more accurate, or both faster and more accurate results, thus presenting three goals and directions.
- **The authors of FNO categorize the AI for PDE methods into three types: Finite-dimensional operators, Neural-FEM, and Neural Operators.**
    - **Finite-dimensional operators** can be understood as neural networks parameterizing an operator in a finite-dimensional space, where both the input and output are in a finite-dimensional Euclidean space. This means that they are attempting to approximate a function. For instance, many current problems in fluid super-resolution are about approximating finite-dimensional operators.
    - The term **'Neural-FEM'** might be an original term coined by the author, referring broadly to models like **PINNs (Physics-Informed Neural Networks).** These models use a finite set of basis functions to parameterize the solution of equations under specific conditions. Using neural networks to approximate solutions under specific conditions inevitably leads to a problem: if the parameters or initial conditions of the equation change, the network must be retrained. This means that for PINNs, the training process is essentially the solving process.

Neural Operators

We consider a class of elliptic boundary value problems, which can be formulated as,

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x), \quad x \in D \tag{1}$$

$$u(x) = 0, \quad x \in \partial D \tag{2}$$

In the equation, u is the solution we are looking for, $f(x)$ is the known source term, and the coefficient function $a(x)$ is also known, which is strictly positive, belonging to a set A, that is, $a(x) > 0, a \in A$, where A is the set of admissible coefficients. We are interested in the operator that maps the source term to the solution, and this operator does not depend on the particular function $a(x)$.

$$(\mathcal{L}_a u)(x) = f(x), \quad x \in D \tag{3}$$

$$u(x) = 0, \quad x \in \partial D \tag{4}$$

Here, $\mathcal{L}_a$ is defined as $-div(a\ div\cdot)$ and represents an elliptic differential operator denoted by L.

One method for finding the analytical solution of partial differential equations is called the Green's function method, which defines a kernel function G, mapping from the domain $D \times D$ to $\mathbb{R}$, and is used to express the solution u throughout the domain D as

$$u(x) = \int_D G_a(x, y) f(y) dy \tag{5}$$

The Green's function G related to the domain is a kernel function, which is a fundamental solution to the differential equation. For example, in the domain $[0, 1] \times [0, 1]$, for a point source located at $(0.5, 0.5)$ with intensity 1, the function $u(0.5)$ is equivalent to the Dirac delta function δ at $(0.5, 0)$ with intensity equal to 0 . **The Green's function $G(0.5, 0.5)$ represents the response**. If the source is not a point but a distributed source, then the solution is a superposition of the responses induced by each point source, which is represented by the integral of the Green's function over the domain D.

In the weak formulation, the integral of the Green's function G with the operator $\mathcal{L}_a G(x, \cdot) = \delta_x$ is equal to the Dirac delta function at x, denoted as δ_x. This represents the weak solution equivalent to the Dirac delta measure, and it is a fundamental concept in the theory of distributions that cannot be represented by a classical function. The Dirac delta function is not a function in the traditional sense but rather a distribution that assigns a measure to a set, which is concentrated at a single point, and its integral over the whole space is equal to one.

Therefore, we have

$$(\mathcal{L}_a u)(x) = \int_D (\mathcal{L}_a G(x, \cdot))(y) f(y) dy \tag{6}$$

$$= \int_D \delta_x(y) f(y) dy \tag{7}$$

$$= f(x) \tag{8}$$

The neural network method solves the PDE by learning an approximate mapping σ. This mapping transforms the combination of the input function and the parameters into the solution. **This leads to a question: can the neural network learn the mapping that is not explicit in traditional numerical methods?** Can it also learn a mapping that represents an infinite-dimensional operator with a finite-dimensional neural network? This is what we hope to explore and verify by introducing neural networks into the solution process of PDEs.

$$v_{t+1}(x) = \sigma(W_t v_t(x) + \int_D k_\phi(x, y, a(x), a(y)) v_t(y) f(y) dy) \tag{9}$$

In the study of a particular neural network, we revisited an old mathematical concept presented under a new name, which at first seemed innovative. For instance, while examining a four-layer ResNet, we encountered the term 'Fourier series', which actually referred to a three-dimensional Fourier transform, essentially the same as the zero-filled discrete Fourier transform. Similarly, the 'Fourier Neural Operator' has been introduced as a rebranded name for a ResNet. These new terms might suggest innovative notations or concepts, but they merely reinterpret existing mathematical operations. **For example, the**

operator σ represents the ReLU function, κ_ϕ is a learned kernel function, and $v_t(y)$ denotes the output of the previous layer or the initial value, where $f(y)$ is the function we are attempting to approximate. Here, ν_x represents a Borel measure, and in this context, $\nu_x(dy) = dy$, which implies that we are integrating with respect to the Lebesgue measure over the domain.

In a study, a neural network was used to approximate the solution of partial differential equations, leveraging the power of modern GPU computing to greatly speed up the process, achieving both efficiency and accuracy. Unlike traditional methods, this neural network-based approach does not require a mesh and can handle complex geometries and boundary conditions, which often slows down the inference process in physics-informed neural networks (PINNs), making their training very time-consuming.

It's important to note that traditional numerical methods for solving PDEs are not suitable for high-dimensional problems due to the curse of dimensionality, which also affects the performance of these traditional methods. This is why general-purpose neural network-based approaches like the one proposed, which approximate a functional mapping from the source term $f(a, g)$ to the solution u, are advantageous for tackling high-dimensional problems.

For the initial condition of the problem, we assume $v_0(x) = f(x)$, with $W = 0$, and the final solution u is obtained by applying the Green's function:

$$G(x, y) = \frac{1}{2}(x + y - |y - x|) - xy \tag{10}$$

Furthermore, the Gaussian Process Regression (GPR) is used, with a kernel that assumes a 2048-dimensional feature space and a prior distribution $f_j \sim \mathcal{N}(0, (-\Delta + I)^{-1})$.

Once more, we find that a five-layer network architecture is mentioned, comprising an encoder and a decoder. This setup utilizes channel space, which is akin to a feature space in neural network terminology.

The functions are defined as follows:

$$v_0(x) = P(x, a(x), \nabla a(x)) + p \tag{11}$$

$$v_{t+1}(x) = \sigma\left(Wv_t(x) + \int_{B(x,r)} \kappa_\phi(x, y, a(x), a(y))v_t(y)dy\right) \tag{12}$$

$$u(x) = Qv_T(x) + q \tag{13}$$

The solution is then expressed by the composite function:

$$u = Q(K_l \circ \sigma \circ \cdots \circ \sigma \circ K_0)Pv \tag{14}$$

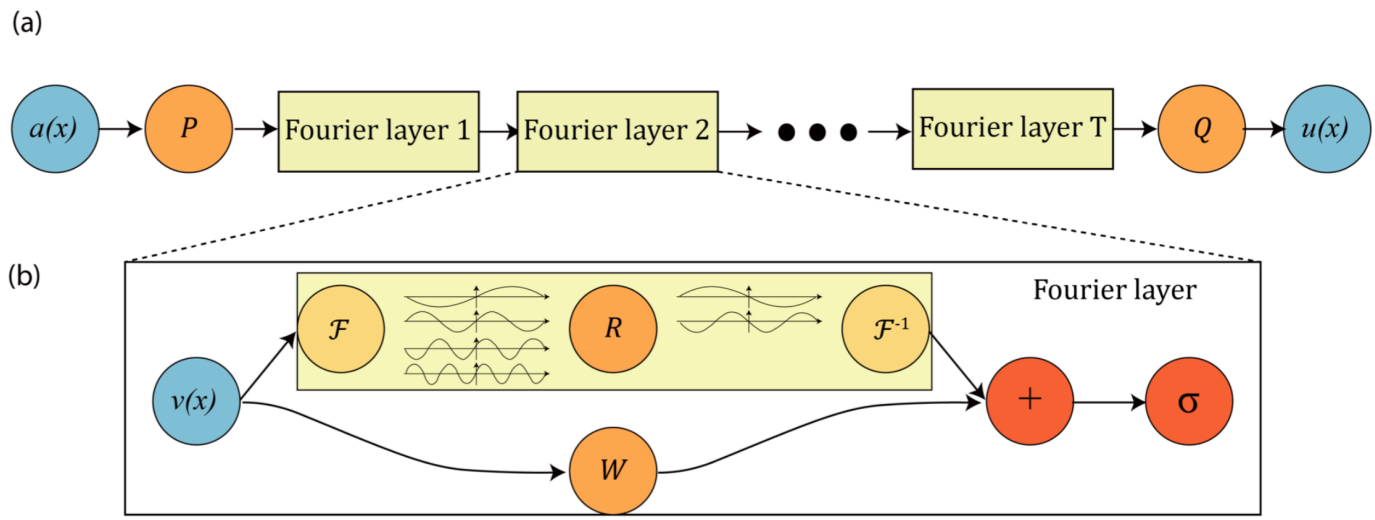
This highlights the key difference between traditional operator-based methods and neural network-based methods like the Fourier Neural Operator, which is designed to approximate these kinds of operators.

The integral kernel is expressed as:

$$\int_D \kappa_\phi(x, y, a(x), a(y))v_t(y)v_x(dy) \tag{15}$$

It is worth mentioning that other neural network approaches, such as **Graph Neural Operator, Multipole Neural Operator, Low-rank Operator, and Fourier Neural Operator**, have been proposed. The latter, in particular, has been shown to be a highly efficient and accurate solution, demonstrating the potential of the Fourier Neural Operator as a powerful tool in the approximation of solutions to PDEs.

Fourier Neural Operator



When designing neural network-based algorithms to approximate solutions of partial differential equations (PDEs), the coefficient function $a(x)$ is often assumed to be known. This assumption simplifies the problem to approximating the mapping from the coefficient function to the solution $u(x)$. Traditional approaches involve learning a global operator Q, while the proposed neural network approach focuses on learning a local operator. This local operator is represented by a neural network that takes the form of a "Fourier Layer", which operates on the local features of $u(x)$ to capture the solution's local behavior. This method differs significantly from traditional Fourier analysis and the global Fourier operators used in standard PDE solvers. Instead, the approach is similar to applying local Fourier filters, comparable to convolutional neural network (CNN) filters but specialized for PDEs.

The algorithm is based on the approximation of local operators, allowing the neural network to learn a "Fourier Operator" for PDEs. This operator is more general than the global Fourier filters applied in traditional methods and can adapt to different local behaviors of the solution.

The Fourier Operator is defined as follows:

$$(K(a;\phi)u)(x) := \int_D k(x,y,a(x),a(y);\phi)u(y)dy, \quad \forall x \in D \tag{16}$$

The integral transform is then inverted to obtain:

$$(JFf)_j(k) = \int_D f_j(x)e^{-2\pi i\langle x,k\rangle}dx, \quad (J^{-1}f)_j(x) = \int_D f_j(k)e^{2\pi i\langle x,k\rangle}dk \tag{17}$$

And applied as:

$$(K(a;\phi)u)(x) = J^{-1}((J(\kappa_\phi)\cdot J(u)))(x), \quad \forall x \in D \tag{18}$$

Furthermore, the Fourier transform is applied to local patches:

$$(K(\phi)u)(x) = J^{-1}(R_\phi \cdot (JFu))(x) \quad \forall x \in D \tag{19}$$

Finally, the iterative process for the neural network-based solution approximation is given by:

$$v_{t+1}(x) := \sigma(Wv_t(x) + (K(a;\phi)u)(x)), \quad \forall x \in D \tag{20}$$

This method allows for a flexible and adaptive approximation of local behaviors of the solution, which is not possible with traditional global operators.

Experiments

In this research, we tackle **the classic Burgers' equation, Darcy Flow, and Navier-Stokes equations using neural networks. The aim is not to directly solve these equations but rather to learn the mappings that represent their solutions.**

For the **Burgers' equation***, the goal is to approximate the complex dynamics that arise. The equation is given as:

$$\frac{\partial u(x,t)}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2(x,t)}{2}\right) = \nu \frac{\partial^2 u(x,t)}{\partial x^2}, \quad x \in (0,1), t \in (0,1) \tag{21}$$

$$u(x,0) = u_0(x), \quad x \in (0,1) \tag{22}$$

The neural network approximates the function $u_0 \mapsto u(\cdot, 1)$.

For the **Darcy Flow**, the equation is given as:

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x), \quad x \in (0,1)^2 \tag{23}$$

$$u(x) = 0, \quad x \in \partial(0,1)^2 \tag{24}$$

The aim is to learn the mapping from the permeability a to the velocity field u_0.

For the **Navier-Stokes equations**, the objective is to approximate the fluid's velocity field that develops over a period of 10 seconds, in a domain that spans $(0,1)^2 \times (0,10]$ to $(0,1)^2 \times (10,T]$ with small viscosity values such as 10^{-3}, 10^{-4}, 10^{-5}.

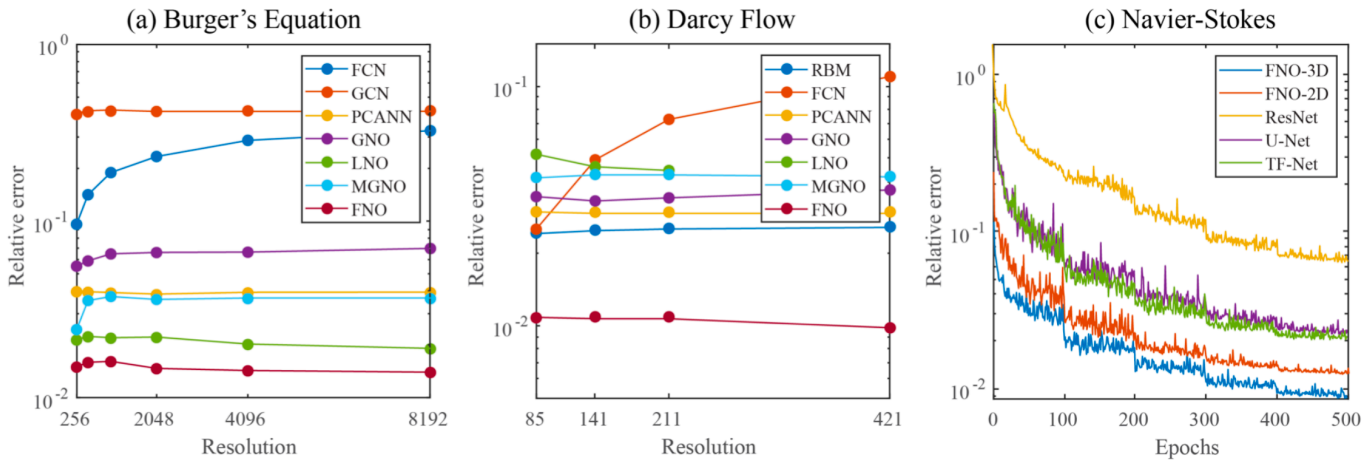
The equations are:

$$\frac{\partial u(x,t)}{\partial t} + u(x,t) \cdot \nabla u(x,t) = \nu \Delta u(x,t) + f(x), \quad x \in (0,1)^2, t \in (0,T] \tag{25}$$

$$\nabla \cdot u(x,t) = 0, \quad x \in (0,1)^2, t \in (0,T] \tag{26}$$

$$u(x,0) = u_0(x), \quad x \in (0,1)^2 \tag{27}$$

The study establishes a benchmark for the neural network's performance.



Config	Parameters	Time per epoch	$\nu = 1e{-}3$ $T = 50$ $N = 1000$	$\nu = 1e{-}4$ $T = 30$ $N = 1000$	$\nu = 1e{-}4$ $T = 30$ $N = 10000$	$\nu = 1e{-}5$ $T = 20$ $N = 1000$
FNO-3D	$6,558,537$	$38.99s$	**0.0086**	0.1918	**0.0820**	0.1893
FNO-2D	$414,517$	$127.80s$	0.0128	**0.1559**	0.0834	**0.1556**
U-Net	$24,950,491$	$48.67s$	0.0245	0.2051	0.1190	0.1982
TF-Net	$7,451,724$	$47.21s$	0.0225	0.2253	0.1168	0.2268
ResNet	$266,641$	$78.47s$	0.0701	0.2871	0.2311	0.2753

It is worth mentioning that the author used the FNO-3D model to train on data of size 64x64x20, resulting in outputs of size 256x256x80, achieving spatio-temporal super-resolution. For the Navier-Stokes equation problems at a fixed resolution of 64x64, it can be seen from the figure below that FNO achieved excellent performance.